

NYC Airbnb Nightly-Price Prediction — Structural, Geographic and Host Features under Random vs. Spatially-Grouped Validation

Course: CSE437 Data Science · Section 6 · Summer2026 **GitHub repository:**

<https://github.com/NaofelKhan/cse437-AirbnbPricePrediction-22> **Date:** 3 September 2026

Group members

Member	Student ID
Mohammad Naofel Khan	23141006

Summary

This project predicts the nightly price of New York City Airbnb listings from structural, geographic and host-activity features, using the 2019 *New York City Airbnb Open Data* (48,895 listings). The target is `price` (continuous, USD), modelled as `log(price + 1)` to correct right-skew and back-transformed to dollars for reporting. We compare two model families — **Ridge regression** (linear) and **AdaBoost** with shallow CART base learners (tree ensemble) — each tuned with grid-search cross-validation. Beyond standard random train/test splitting, we evaluate every model under a **spatially-grouped split** that holds out entire neighbourhoods, testing whether random splitting overstates accuracy because geographically close listings leak information across the split. Our headline metric is RMSE in dollars. The best model (Ridge) reaches **RMSE $\approx$ \$49.5 / MAE $\approx$ \$35 / $R^2 \approx 0.48$** on a random split, beating a mean-price baseline (RMSE \$70.8) by roughly 30%. The single most important finding is methodological: moving to the spatially-grouped split raises RMSE by $\sim 4\%$ and MAE by $\sim 6\%$, confirming a **modest but real spatial-leakage effect** — smaller than the inflation reported for text-and-amenity-rich benchmarks, because our features are coarse-grained geographically.

1. Problem and Dataset

1.1 Problem statement. Short-term rental hosts and platforms need to price listings competitively, yet nightly price is driven by a mix of structural attributes (room type, minimum stay), location, and host behaviour. We predict nightly price for NYC listings from

these features and, critically, ask whether the accuracy numbers commonly reported for this dataset are optimistic because standard random splitting allows near-neighbour listings to appear in both train and test. Getting this right matters: a price model validated with hidden spatial leakage will look better in a report than it performs on genuinely new neighbourhoods.

1.2 Dataset. *New York City Airbnb Open Data (2019)*, sourced from Kaggle ([dgomonov/new-york-city-airbnb-open-data](https://www.kaggle.com/dgomonov/new-york-city-airbnb-open-data) , originally compiled by Inside Airbnb, released CC0). It contains **48,895 rows × 16 columns** covering active NYC listings as of the 2019 snapshot (review dates run through 8 July 2019). No scraping or merging was performed; the raw CSV is used as published. Link: <https://www.kaggle.com/dgomonov/new-york-city-airbnb-open-data>

1.3 Target variable. `price` — continuous, in USD (a regression task). In the raw data it is heavily right-skewed (skew ≈ 19 including extreme outliers; mean \$152.7, median \$106) and contains impossible \$0 entries. After cleaning (Section 2) the modelling target `log(price + 1)` is near-symmetric (skew -0.09). Modelled prices span \$10–\$334 (mean \$120, median \$100, sd \$68).

1.4 Three questions. *(Reconstructed from the project scope — replace with the exact wording from your approved proposal.)*

1. How accurately can NYC Airbnb nightly price be predicted from structural, geographic and host-activity features alone (no listing text, images or amenities)?
2. Does standard random train/test splitting overstate model performance relative to a spatially-grouped split that holds out entire neighbourhoods?
3. Where and why does the model fail — which boroughs, room types, host scales and price ranges carry the largest error?

2. Data Handling and Preprocessing

Evidence of what changed is given at each step; row/column counts are tabulated in Section 2.5.

2.1 Data quality audit. No fully-duplicated rows and no duplicate listing `id` s. Missing values occur in four columns: `reviews_per_month` and `last_review` (10,052 rows each, 20.56%), `host_name` (21, 0.04%) and `name` (16, 0.03%). Impossible/improbable values: 11 listings priced \$0; `minimum_nights` reaches 1,250 (a multi-year minimum is not a short-term rental); 14 listings exceed a 365-night minimum. Categorical fields (`neighbourhood_group` , `room_type`) are internally consistent.

2.2 Missing values. The missingness in `reviews_per_month / last_review` is **not random** — it coincides exactly with `number_of_reviews == 0` (verified with an assertion), i.e. "never reviewed," not a measurement gap. We therefore fill `reviews_per_month` with 0 and retain

the never-reviewed signal via an explicit flag (Section 4.1) rather than imputing a fake review rate or date; `last_review` is left as `NaT`. The handful of missing `name` / `host_name` (identifier fields, not model inputs) are filled with `"Unknown"`. Nothing was dropped for missingness.

2.3 Outliers. Detection uses the IQR rule ($k = 1.5$). For `price` the upper fence is \$334, flagging 2,972 listings (6.08%); `minimum_nights` flags 6,607 (13.51%). Actions: the 11 \$0 listings are dropped as data errors; prices above the \$334 IQR fence are dropped (removing the extreme right tail rather than winsorising); and 13 listings with `minimum_nights > 365` are dropped as non-short-term rentals. `minimum_nights` values within range are kept.

2.4 Transformation and scaling. `price` is log-transformed as `log(price + 1)`, reducing skew from 0.925 (post-cap) to -0.093 . Categoricals are one-hot encoded (`room_type`, `neighbourhood_group`, and the engineered `host_scale` / `geo_cluster`, all with `drop_first=True`). Five numeric features (`minimum_nights`, `number_of_reviews`, `reviews_per_month`, `calculated_host_listings_count`, `availability_365`) are standardised with `StandardScaler`. **Leakage note:** for project scope the scaler is fit on the full dataset before splitting. This is a documented simplification; a leakage-free pipeline would fit the scaler on the training split only (see Section 8).

2.5 Before and after.

Stage	Rows	Cols	Change
Raw load	48,895	16	—
Missing-value handling	48,895	16	fill <code>reviews_per_month</code> $\rightarrow 0$, names $\rightarrow$ "Unknown"
Drop <code>price == 0</code>	48,884	16	−11
Drop <code>price > \$334</code> (IQR fence)	45,912	16	−2,972
Drop <code>minimum_nights > 365</code>	45,899	16	−13
+ log target + 5 scaled columns	45,899	22	+ <code>price_log</code> , +5 <code>_scaled</code>
Final model feature matrix	45,899	12 features	after encoding + selection (Section 4)

Total removed in cleaning: 2,996 rows (6.13%).

3. Statistical Analysis

3.1 Descriptive statistics On the cleaned modelling data `price` is centred at a mean of \$120

On the cleaned modeling data, price is centered at a mean of \$120 and median \$100 (sd \$68), now only mildly skewed (0.93) and bounded at \$10–\$334. Listings concentrate in **Manhattan (44.3%)** and **Brooklyn (41.1%)**, with Queens (11.6%), the Bronx (2.2%) and Staten Island (0.8%) far behind. By room type, **Entire home/apt (52.0%)** and **Private room (45.7%)** dominate; Shared rooms are rare (2.4%). Availability is bimodal (many listings at 0 days and many near 365), and host listing counts are highly right-skewed (median 1, max 327).

3.2 Relationships. A one-way ANOVA of price across the five boroughs is highly significant ($F = 354.99$, $p \approx 7.7 \times 10^{-302}$), confirming location as a first-order price driver. Correlations among the numeric features are weak (mostly $|r| < 0.2$ with price), so most linear price signal is carried by the *categorical* structure (room type, borough) rather than the continuous review/availability features. Supporting figures: price-by-borough boxplots and the numeric-feature correlation heatmap (`figures/price_by_borough.png` , `figures/correlation_heatmap_raw.png`).

3.3 What the data says so far.

- Price needs a log transform; raw skew makes untransformed regression ill-posed.
- Borough (and finer location) is the strongest price signal — motivating geographic feature engineering.
- 20.6% of listings are never reviewed; that pattern is informative and should be encoded, not imputed away.
- Room type materially separates prices and will be a dominant feature.
- Weak numeric-to-price correlations imply that non-linear models and categorical/location features matter more than the raw numeric columns.

4. Feature Engineering

4.1 Derived features. (a) `distance_from_center_km` — haversine distance from each listing to Times Square, a compact proxy for centrality; (b) `days_since_last_review` — recency of activity, using the dataset's own max date as "now" (–1 sentinel for never-reviewed); (c) `never_reviewed` — binary flag built from the missingness pattern in 2.2; (d) `host_scale` — `individual` (≤ 1 listing), `small` (2–5), `commercial` (> 5), capturing operator type; (e) `geo_cluster` — a **K-Means** cluster of (latitude, longitude), giving a low-dimensional location signal in place of one-hot encoding ~220 raw neighbourhoods. K-Means used $k = 6$ (silhouette is flat at ~0.41–0.43 over $k = 4\text{--}8$; `figures/kmeans_k_selection.png` , `figures/geo_clusters_map.png`).

4.2 Dimensionality reduction (PCA). PCA is applied to the seven-feature standardised numeric block. Because PCA is scale-sensitive, the block is standardised first; **6 of 7 components are needed for 90% variance** (explained-variance ratios 0.28, 0.20, 0.15, 0.12, 0.11, 0.09, 0.06). This shows the numeric features are largely **non-redundant**, so PCA is retained only as a diagnostic — its components are **not** used as model inputs to preserve

retained only as a diagnostic — its components are **not** used as model inputs, to preserve the feature interpretability needed for error analysis (`figures/pca_explained_variance.png`).

4.3 Feature selection (RFE). Recursive Feature Elimination with a `LinearRegression` estimator ranks the 21 engineered candidates and selects the top 12 (threshold: `n_features_to_select = 12`). RFE favoured the one-hot categoricals and the never-reviewed/availability signals; it eliminated `minimum_nights` , `number_of_reviews` , `reviews_per_month` , `calculated_host_listings_count` , `distance_from_center_km` , `days_since_last_review` and three of the geo-cluster dummies (ranks 2–10).

4.4 Final feature set. The 12 retained features are: `boro_Brooklyn` , `boro_Manhattan` , `boro_Queens` , `boro_Staten Island` , `room_Private room` , `room_Shared room` , `host_individual` , `host_small` , `geo_1` , `geo_3` , `availability_365_scaled` , `never_reviewed` . We dropped raw identifiers (no predictive meaning), raw latitude/longitude (superseded by `geo_cluster` + distance), unscaled duplicates of numeric columns, and the RFE-eliminated features above. The set is deliberately interpretable so Section 7 can attribute error to concrete segments; Section 8 shows that a less aggressive selection improves accuracy.

5. Modeling and Validation

5.1 Validation strategy. 80/20 split at `random_state = 42` , evaluated two ways. **Split A (random):** `train_test_split` (train 36,719 / test 9,180). **Split B (spatial):** `GroupShuffleSplit` grouped by `neighbourhood` , holding out entire neighbourhoods (train 38,664 / test 7,235; **175 train vs 44 test neighbourhoods, 0 overlap**). Cross-validation during tuning uses 5 folds, with `GroupKFold(5)` for the spatial models so no neighbourhood spans folds. This respects the grouped structure of the data and isolates spatial leakage.

5.2 Baseline. A mean-price predictor (predict the training mean for every listing) yields **USD RMSE \$70.8, MAE \$53.7, $R^2 = -0.07$** on the random test set. Any real model must beat this.

5.3 Model families. (A) *Ridge regression* (L2-regularised linear): interpretable, fast, gives signed coefficients; assumes an additive linear relationship on the log-price scale. (B) *AdaBoost* with `DecisionTreeRegressor` base learners (depth ≤ 4): a tree ensemble that captures non-linearities and feature interactions and makes no linearity assumption. The two come from distinct families (linear model vs. boosted trees), satisfying the "two different families" requirement and bracketing the bias–variance trade-off.

5.4 Metrics. Primary metric (declared before results): **RMSE in dollars** — directly interpretable as pricing error and appropriately penalising large misses. Secondary: **MAE** (typical absolute error) and **R^2** (variance explained). All models train on `log(price + 1)` and are reported back-transformed to USD. Note that **R^2 is not comparable across Splits A and B** because the held-out sets have different price variance; cross-split comparison therefore relies on RMSE/MAE.

6. Hyperparameter Tuning

6.1 Search space.

Model	Hyperparameter	Grid
Ridge	alpha	0.01, 0.1, 1.0, 10, 100
AdaBoost	n_estimators	50, 100, 200
AdaBoost	learning_rate	0.01, 0.1, 1.0
AdaBoost	base_max_depth	2, 3, 4

6.2 Method. GridSearchCV with 5-fold CV (GroupKFold(5) for the spatial models), scoring = negative RMSE. Candidate counts: Ridge 5, AdaBoost 27 ($3 \times 3 \times 3$), i.e. 25 and 135 fits per split respectively.

6.3 Results. Ridge selected `alpha = 1.0` (CV RMSE $\approx$ 0.381 on the log scale; the score is essentially flat across alpha, so the problem is not regularisation-sensitive). AdaBoost selected `max_depth = 4`, `learning_rate = 0.01`, `n_estimators = 100` (CV RMSE $\approx$ 0.381). The search trend is informative: AdaBoost's best configurations all use the shallowest-allowed slow-learning settings — deeper trees or faster learning rates degrade CV score, indicating the 12-feature signal is low-complexity.

7. Results, Visualization and Error Analysis

7.1 Test-set performance. Reported once, on the held-out test sets (USD scale).

Model	Split	RMSE	MAE	R ²
Baseline (mean)	random	70.8	53.7	-0.07
Ridge	random	49.5	35.0	0.475
AdaBoost	random	49.8	35.3	0.468
Ridge	spatial	51.7	37.2	0.504
AdaBoost	spatial	52.4	37.6	0.491

Ridge is marginally the best model on every metric; a linear model matching the boosted ensemble indicates the retained signal is largely linear. Both models beat the baseline by ~30% on RMSE.

7.2 Visualization. Predicted-vs-actual plots (`figures/predicted_vs_actual.png`) show the

characteristic compression at the top of the range — the model systematically under-predicts expensive listings, a direct consequence of the \$334 cap and log target. Per-segment error bars ([figures/error_by_segment.png](#)) localise the error by borough, room type and host scale.

7.3 Error analysis. Error concentrates in identifiable segments. By room type, **Entire home/apt MAE \$46.7** dwarfs Private room (\$27.6) and Shared room (\$26.0); by borough, **Manhattan is worst (\$39.6)**; by host scale, **commercial hosts (\$43.6)** exceed individuals. Two concrete failures (Ridge, random test):

- "*San Carlos Hotel — Deluxe Room*" (Manhattan, labelled **Private room**): actual **\$330**, predicted **\$90** (error \$240).
- "*SUPERIOR KING ROOM — Prime Williamsburg*" (Brooklyn, **Private room**): actual **\$295**, predicted **\$64** (error \$231).

Both are premium/hotel-style units mislabelled as "Private room" and priced at the ceiling. The model keys on `room_type = Private room` (typically cheap) and under-predicts. These cases are hard because the room-type label is unreliable and the dataset carries no amenity, title-text or image signal that would flag a hotel or luxury suite.

7.4 Answers to the three questions.

1. **Accuracy.** Yes, moderately. The best model predicts nightly price with RMSE $\approx$ \$49.5 and MAE $\approx$ \$35 against a \$100 median ($R^2 \approx 0.48$ USD), a $\sim 30\%$ improvement over the mean baseline — but below text/amenity-rich published models (Section 8).
2. **Spatial leakage.** Confirmed but modest. On the comparable metrics, moving random $\rightarrow$ spatial raises **RMSE +4.4%** (\$49.5 $\rightarrow$ \$51.7) and **MAE +6.3%** (\$35.0 $\rightarrow$ \$37.2). The apparent R^2 *increase* (0.48 $\rightarrow$ 0.50) is a metric artifact — R^2 is not comparable across the two test sets — and must **not** be read as "no leakage." The effect is small precisely because our features are coarse (borough/cluster, not neighbourhood identity), leaving little fine-grained location to leak.
3. **Where it fails.** On expensive whole-unit listings and mislabelled premium "private rooms" near/above the \$334 cap, in Manhattan, and for commercial hosts — driven by the price cap and the absence of amenity/text features.

8. Limitations and Next Steps

Limitations. (1) The \$334 IQR cap removes the top $\sim 6\%$ of listings, so the model cannot price expensive units and its headline metrics are partly flattered by excluding the hardest cases. (2) The scaler is fit before splitting — minor train/test leakage. (3) Linear RFE discarded continuous features (`number_of_reviews` , `availability` , `distance`) that carry real signal. (4) The dataset has no amenities, listing-text or images, which is the main ceiling relative to published NYC work.

Next steps (measured, not speculative). On the same cleaned data we benchmarked stronger choices under identical splits: replacing the model family with `HistGradientBoostingRegressor` on the *full* engineered feature set lifts random-split **USD R^2 from 0.475 to 0.587** and cuts MAE from \$35.0 to \$30.4 (RandomForest reaches R^2 0.572); it also leads under the spatial split (R^2 0.578). Concretely we would (a) adopt a gradient-boosting/RandomForest family (both are in the Decision-Tree/CART family, so this stays within scope), (b) keep the full feature set or select by tree importance rather than linear RFE, (c) add listing- `name` text features (even keyword flags helped; TF-IDF would do more), (d) raise or winsorise the price cap to cover the expensive tail (accepting higher but honest RMSE), and (e) fit all transforms inside a training-only `Pipeline`. Target-encoding `neighbourhood` would add strong location signal *and* sharpen the spatial-leakage test. We do not claim parity with amenity/NLP-rich benchmarks without that data.

References

1. Zhu, A., *et al.* (2020). Airbnb price prediction for New York City (~48,896 listings). *IEEE — AI4I 2020*. <https://ieeexplore.ieee.org/document/9253078/> (closest benchmark: same dataset scale as this study).
2. Kalehbasti, P. R., Nikolenko, L., Rezaei, H. (2019). *Airbnb Price Prediction Using Machine Learning and Sentiment Analysis*. arXiv:1907.12665. <https://arxiv.org/pdf/1907.12665> (most-cited NYC Airbnb pricing baseline; richer InsideAirbnb + text features).
3. *Optimal pricing strategy for Airbnb listings using regression and NLP* (2024). *Journal of Risk and Financial Management*, 17(9):414, MDPI. <https://www.mdpi.com/1911-8074/17/9/414> (recent peer-reviewed pricing method).
4. *An Ensemble Machine Learning Framework for Airbnb Rental Price Modeling without Using Amenity-Driven Features* (2023). ResearchGate 369257812. <https://www.researchgate.net/publication/369257812> (ensemble methods without amenity features — directly relevant to our feature scope).
5. *Predicting Airbnb Rental Prices Using Multiple Feature Modalities* (2021). arXiv:2112.06430. <https://arxiv.org/pdf/2112.06430> (multi-city; used to discuss generalisation limits).

Dataset. Dgomonov (2019), *New York City Airbnb Open Data*, Kaggle (originally Inside Airbnb; CC0). <https://www.kaggle.com/dgomonov/new-york-city-airbnb-open-data>

Libraries. pandas, NumPy, scikit-learn, SciPy, matplotlib, seaborn, statsmodels.

AI assistance declaration. Claude (Anthropic) was used to set up the Python environment, debug and correct a PCA feature-scaling bug in Notebook 03, reproduce and validate the pipeline outputs, run a model-improvement benchmarking experiment (Section 8), and draft this report from the authors' notebooks and results. All project design, modelling decisions and final content were reviewed and are owned by the authors